

1. 油瓶分油问题

将问题向量化，以 (x, y) 表示 8 斤瓶中的油量 x 和 6 斤瓶中的油量 y

则问题需要从起始态 $(0, 0)$ 到达目标态 $(4, 0)$

$(x, y) \ \&\& \ x < 8 \rightarrow (8, y)$

$(x, y) \ \&\& \ y < 6 \rightarrow (x, 6)$

$(x, y) \ \&\& \ x > 0 \rightarrow (0, y)$

$(x, y) \ \&\& \ y > 0 \rightarrow (x, 0)$

$(x, y) \ \&\& \ x > 0 \ \&\& \ x+y \leq 6 \rightarrow (0, x+y)$

$(x, y) \ \&\& \ y > 0 \ \&\& \ x+y \leq 8 \rightarrow (x+y, 0)$

$(x, y) \ \&\& \ x > 0 \ \&\& \ x+y > 6 \rightarrow (8, x+y-8)$

$(x, y) \ \&\& \ y > 0 \ \&\& \ x+y > 6 \rightarrow (x+y-6, 6)$

根据以上规则，可得该问题的两种解法：

一： $(0, 0) \rightarrow (0, 6) \rightarrow (6, 0) \rightarrow (6, 6) \rightarrow (8, 4) \rightarrow (0, 4) \rightarrow (4, 0)$

二： $(0, 0) \rightarrow (8, 0) \rightarrow (2, 6) \rightarrow (2, 0) \rightarrow (0, 2) \rightarrow (8, 2) \rightarrow (4, 6) \rightarrow (4, 0)$

2. 猴子与香蕉问题

• 状态表示：

用一个四元表列 (W, x, Y, z) 来表示这个问题的状态，

其中

W —猴子的水平位置

x —当猴子在箱子顶上时取 $x=1$ ；否则取 $x=0$

Y —箱子的水平位置

z —当猴子摘到香蕉时取 $z=1$ ；否则取 $z=0$

• 变换规则：

(1) 猴子走到水平位置 U

$(W, 0, Y, z) \rightarrow (U, 0, Y, z)$

(2) 猴子把箱子推到水平位置 V

$(W, 0, W, z) \rightarrow (V, 0, V, z)$

(3) 猴子爬上箱顶

$(W, 0, W, z) \rightarrow (W, 1, W, z)$

(4) 猴子摘到香蕉

$(c, 1, c, 0) \rightarrow (c, 1, c, 1)$

- 搜索方式：使用 DFS 求解，直至找到满足问题的解

3. 八数码问题

- 状态表示：

用 3×3 的矩阵表示九宫格，0 表示空格

- 变换规则：

0 上下左右移动并与相应位置上的数字交换位置

- 搜索方式：

使用启发式搜索，该问题的评估函数 $f(n) = g(n) + h(n)$ 中， $g(n)$ 可以选用从起始态变换到当前状态移动的步数， $h(n)$ 可以用当前状态距离目标态位置不同的数字（或空格）个数表示。

4. 广度优先算法的利弊

(1) 在产生新的子结点时，深度越小的结点越先得到扩展，即先产生它的子结点。为使算法便于实现，存放结点的数据库一般用队列的结构。

(2) 无论问题性质如何不同，利用广度优先搜索法解题的基本算法是相同的，但数据库中每一结点内容，产生式规则，根据不同的问题，有不同的内容和结构，就是同一问题也可以有不同的表示方法。

(3) 当结点到根结点的费用（有的书称为耗散值）和结点的深度成正比时，特别是当每一结点到根结点的费用等于深度时，用广度优先法得到的解是最优解，但如果不成正比，则得到的解不一定是最优解。这一类问题要求出最优解，一种方法是使用后面要介绍的其他方法求解，另外一种方法是改进前面深度（或广度）优先搜索算法：找到一个目标后，不是立即退出，而是记录下目标结点的路径和费用，如果有多个目标结点，就加以比较，留下较优的结点。把所有可能的路径都搜索完后，才输出记录的最优路径。

(4) 广度优先搜索算法，一般需要存储产生的所有结点，占的存储空间要比深

度优先大得多，因此程序设计中，必须考虑溢出和节省内存空间得问题。

(5) 广度优先搜索法一般无回溯操作，即入栈和出栈的操作，所以运行速度比深度优先搜索算法法要快些。

5. 深度优先搜索的利弊

(1) 无论问题的内容和性质以及求解要求如何不同，它们的程序结构都是相同的，不相同的仅仅是存储结点数据结构和产生规则以及输出要求。

(2) 深度优先搜索法有递归以及非递归两种设计方法。一般的，当搜索深度较小、问题递归方式比较明显时，用递归方法设计好，它可以使得程序结构更简捷易懂。当搜索深度较大时，当数据量较大时，由于系统堆栈容量的限制，递归容易产生溢出，用非递归方法设计比较好。

(3) 深度优先搜索方法有广义和狭义两种理解。广义的理解是，只要最新产生的结点（即深度最大的结点）先进行扩展的方法，就称为深度优先搜索方法。在这种理解情况下，深度优先搜索算法有全部保留和不全部保留产生的结点的两种情况。而狭义的理解是，仅仅只保留全部产生结点的算法。本书取前一种广义的理解。不保留全部结点的算法属于一般的回溯算法范畴。保留全部结点的算法，实际上是在数据库中产生一个结点之间的搜索树，因此也属于图搜索算法的范畴。

(4) 不保留全部结点的深度优先搜索法，由于把扩展望的结点从数据库中弹出删除，这样，一般在数据库中存储的结点数就是深度值，因此它占用的空间较少，所以，当搜索树的结点较多，用其他方法易产生内存溢出时，深度优先搜索不失为一种有效的算法。

(5) 从输出结果可看出, 深度优先搜索找到的第一个解并不一定是最优解。

6. BFS 求解迷宫



图 3.29 迷宫一例

① $S \rightarrow A \rightarrow B$

② $B \rightarrow C$ $B \rightarrow H$

③ $C \rightarrow F$ $H \rightarrow G$

故 $S \rightarrow A \rightarrow B \rightarrow C \rightarrow F$

7. 什么是 A 算法，什么是 A*算法

- 定义评价函数: $f(n) = g(n) + h(n)$ 对 OPEN 表中的元素按照 f 值, 从小到大进行排列, 每次从 OPEN 表中取出 f 值最小的结点扩展, 这种图搜索算法成为 A 算法。

- A*算法是一种静态路网中求解最短路径最有效的直接搜索方法, 是对 A 算法的估价函数 $f(n) = g(n) + h(n)$ 加上某些限制后得到的一种启发式搜索算法, 也是许多其他问题的常用启发式算法。公式表示为: $f^*(n) = g^*(n) + h^*(n)$, 其中, $f^*(n)$ 是从初始状态经由状态 n 到目标状态的最小代价估计, $g^*(n)$ 是在状态空间中从初始状态到状态 n 的最小代价, $h^*(n)$ 是从状态 n 到目标状态的路径的最小估计代价。(对于路径搜索问题, 状态就是图中的节点, 代价就是距离)。真实 $h(n)$ 的选取: 保证找到最短路径 (最优解的) 条件, 关键在于估价函数 $f(n)$ 的选取 (或者说 $h(n)$ 的选取)。以 $h(n)$ 表达状态 n 到目标状态估计的距离, 那么 $h(n)$ 的选取大致有如下三种情况: 如果 $h(n) < h^*(n)$, 这种情况下, 搜索的点数多, 搜索范围大, 效率低。但能得到最优解。如果 $h(n) = h^*(n)$, 此时的搜索效率是最高的。如果 $h(n) > h^*(n)$, 搜索的点数少, 搜索范围小, 效率高, 但不能保证得到最优解。 [